For more details about tail calls and recursion, see this excellent post.

Most compilers optimize tail calls, by shortcutting the stack frame creation. With tail recursion compilers can go even further, by rewriting the chain calls as a loop.

As for our example, in the case of JavaScript, support for tail call optimization was introduced with ES2015!

To make sure the compiler can optimize the machine code generated for our factorial function, however, we need to slightly change it, to make sure the recursive call is the last operation before returning.

Listing E.6 Factorial function, with tail recursion

```
function factorial(n) {
  if (n < 0) {
    throw new RangeError('n can not be negative');
  } else if (n === 0) {
    return 1;                                          #1
  }

  return n * factorial(n-1);                           #2
}
```

#1 Base case
#2 The recursive call is the last instruction in this function

What the compiler will do to optimize the tail call above is simply translating it into a loop. Turns out, in fact, that tail